



UNIFIED MENTOR · PROJECT REPORT

LegacyCare

Dignified End-of-Life & Funeral Planning Platform

Submitted by	Lakshay Pandit
Program	Unified Mentor – Web Development Internship
Project ID	17815
Technology	MERN stack: MongoDB, Express.js, React, Node.js
Live application	https://legacycare-unified-mentor-project.vercel.app
Source code	github.com/lxkshxyy/LegacyCare-Unified-Mentor-Project
Date	October 2026

Contents

- | | |
|-------------------------------------|-------------------------------|
| 1. Abstract | 10. Security & privacy |
| 2. Introduction & problem statement | 11. UI / UX design |
| 3. Objectives & scope | 12. Application screenshots |
| 4. Requirement analysis | 13. Testing |
| 5. Reference study (efuneral.com) | 14. Deployment |
| 6. System architecture | 15. KPIs |
| 7. Database design | 16. Challenges & solutions |
| 8. Modules & features | 17. Conclusion & future scope |
| 9. API design | |

1 Abstract

LegacyCare is a full-stack web application that lets people **plan their own funeral and last rites in advance**, according to their personal, cultural and religious wishes. A planner records where and how the ceremony should happen, which rituals and officiant they want, music, prayers and customs, and which verified service providers to use, along with an estimated budget. They then nominate a family member or friend who can see the finalized plan when it is needed.

The platform serves four kinds of users: **Planners, Nominees / Family, Service Providers** and **Administrators**. It is built with the MERN stack (MongoDB, Express.js, React 19 and Node.js) and styled with Tailwind CSS. It uses JWT authentication, role-based access control and AES-256-GCM encryption for sensitive fields, and it is deployed on Vercel with MongoDB Atlas.

Outcome: a working, deployed web application with four role-based dashboards, a 7-step plan wizard, a verified provider directory, a request workflow, an admin KPI console, full documentation and 51 passing automated API checks.

2 Introduction & problem statement

Death is one of the few certainties of life, yet very few people in India write down how they want their last rites performed. When a death happens, grieving families have to make dozens of decisions within hours: the cremation or burial ground, the pandit or priest, transport, flowers, rituals and budget. Often they are unsure what the person would have wanted.

The project brief identifies these problems:

- No centralised digital system for advance funeral planning
- Emotional confusion and rushed decisions by families
- Lack of clarity on rituals, service providers and costs
- Difficulty honouring personal and cultural preferences
- Manual coordination with many separate vendors

LegacyCare addresses these with one calm, respectful and secure place to record wishes, compare verified providers, and share everything with the people who will need it.

3 Objectives & scope

Primary objectives

- Let individuals pre-plan funeral and last-rite arrangements
- Respect personal, cultural and religious preferences
- Give access to verified funeral and ritual service providers
- Reduce emotional and logistical burden on families

Secondary objectives

- Promote dignity and transparency in end-of-life planning
- Digitise coordination between families and providers
- Securely store instructions and documents
- Provide budget clarity and service breakdowns

In scope

- Responsive web platform
- Secure planning dashboard
- Provider listings
- Preference documentation & storage
- Nominee / family access management

Out of scope

- Native mobile apps
- Legal will drafting
- Government death-certificate processing
- Emergency medical / ambulance services

4 Requirement analysis

Functional requirements – implementation

Role	Requirement	How LegacyCare implements it
Planner	Secure registration & login	bcrypt-hashed passwords, JWT sessions, role chosen at sign-up
	Plan: location, ritual type, priest/pandit, ceremony instructions	7-step wizard with save-as-draft at every step
	Select funeral agencies, transport, flowers	Pick from the verified directory; price frozen at selection; live budget total
	Upload documents & notes	Multipart upload, type whitelist, 5 MB limit, labels
	Assign nominee; modify anytime	Nominees by email with access toggle; version number + update history

Nominee	Secure authorised access; view plan	Only finalized plans, only for the listed email, only while access is on
	Download instructions; contact providers	Print-to-PDF, authenticated document download, call/email links, service requests
	Reminders & guidance	In-app notifications and an 8-step "What to do now" checklist
Provider	Registration & verification	Status pending → verified/rejected by admin; hidden until verified
	Listings with pricing; availability	CRUD listings with price unit, city, traditions; availability toggle
	Receive and update requests	Inbox with accept / decline / complete plus a note to the family
Admin	Verify users & providers	Approve, reject with reason, suspend, verify identity
	Manage ritual categories & service types	Category manager (ritual / service, tradition, visibility)
	Monitor usage; disputes; ethics	KPI dashboard, dispute resolution, no access to plan contents

Non-functional requirements

Category	Implementation
Security	AES-256-GCM encrypted fields, RBAC, Helmet, rate limiting, CORS rules
Reliability	Mongoose schema validation, MongoDB Atlas managed storage and backups
Usability	Calm palette, serif headings, gentle wording, labelled accessible forms, mobile-first layout
Performance	Vite production build, route-level code splitting, gzip, indexed queries
Compliance	Minimal data collection, user-controlled deletion, admin cannot read private plans

5 Reference study: efuneral.com

The design reference provided was **efuneral.com**, a US funeral pre-planning marketplace. Its patterns were studied and adapted for an Indian, multi-faith audience.

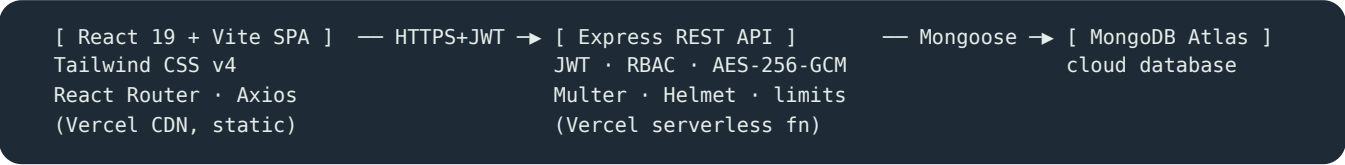
eFuneral pattern

LegacyCare adaptation

Hero: "On Your Terms. At Your Fingertips."	"Plan with dignity. Leave behind clarity, not confusion."
Primary CTA "Find a Provider"	Two CTAs: <i>Start your plan</i> and <i>Find a provider</i>
4-step icon process	Create plan → Choose rituals & providers → Add nominee → Rest easy
"What should I know?" FAQ	FAQ accordion on the home page plus a full FAQ page
"Are you prepared?" free guide	Planning checklist call-to-action
"Become a Partner"	"Become a provider" → provider registration and verification
Minimal, circular imagery	Ivory background, sage & gold accents, circular icon treatments, lotus mark

Beyond the reference: eFuneral has no nominee access, document vault, multi-faith ritual preferences, provider request workflow or admin verification. These are LegacyCare's differentiators and come straight from the project brief.

6 System architecture



Layer	Technology & responsibility
Presentation	React 19, React Router 7, Tailwind CSS 4, lucide-react icons; role-based layouts and protected routes
API	Express 4 with MVC structure: models, controllers, routes and middleware (auth, upload, errors)
Data	MongoDB Atlas via Mongoose 9 with schema validation, indexes and encrypted getters/setters
Hosting	Vercel: static frontend plus a serverless Express function on one domain (no CORS issues)

7 Database design

```
User —< Plan —< Document
|         |— nominees[] { name, email, relation, phone🔒, accessGranted }
|         |— selectedServices[] → Service → User (provider)
|—< Service      (provider listings)
|—< ServiceRequest (plan, requester, provider, service, status, history[])
|—< Dispute · Notification · Feedback
Category (ritual | service)
```

Collection	Key fields
User	name, email (unique), password (bcrypt), role, phone, city, status, isVerified, provider { businessName, licenseNumber, serviceAreas, verificationStatus }
Plan	owner, title, status (draft/finalized), location, disposition, ritual, officiant, ceremony🔒, personalNotes🔒, selectedServices[], budget, nominees[], version, updateHistory[]
Service	provider, title, category, description, price, priceUnit, city, traditions[], isAvailable, isActive
ServiceRequest	plan, requester, provider, service, type (pre-booking/execution), status, providerNote, history[]
Document	plan, owner, label, originalName, storedName (hidden), mimeType, size
Category	name, kind (ritual/service), tradition, description, isActive
Dispute	raisedBy, against, subject, message, priority, status, resolution
Notification / Feedback	user, message, link, read / user, rating (1–5), comment

🔒 = encrypted at rest with AES-256-GCM (stored as enc:v1:iv:tag:cipher).

8 Modules & features

Public website

- Home page modelled on the reference: hero, 4 steps, traditions, features, provider categories, FAQ, CTA
- How it works for planners, nominees and providers
- Provider directory with search, category, city, budget, availability and sort filters
- Login (one-click demo accounts) and role-based registration

Planner

- Overview: completion ring, nominees, services, budget, update count
- 7-step wizard plus review and finalize, with a sticky budget panel
- Plan detail: print/PDF, send requests, move back to draft
- Requests, notifications, feedback and disputes

Nominee / family

- "Shared with me" list of finalized plans
- Read-only plan with documents and provider contacts
- "What to do now" guidance checklist (8 steps)
- Immediate-need service requests to providers

Provider & admin

- Provider: verification banner, stats, listings CRUD, availability toggle, request inbox
- Admin: 5 KPI cards and charts, users, provider verification queue, categories, disputes

9 API design

REST API under `/api`. Every protected route uses `protect` (JWT verification, blocks suspended users) and `authorize(roles)`. Plan routes add ownership and nominee-access checks.

Area	Endpoints
Auth	POST <code>/auth/register</code> · POST <code>/auth/login</code> · GET/PUT <code>/auth/me</code> · PUT <code>/auth/password</code>
Plans	GET/POST <code>/plans</code> · GET <code>/plans/shared</code> · GET/PUT/DELETE <code>/plans/:id</code> · POST <code>/plans/:id/finalize</code> · <code>/reopen</code>
Plan services & nominees	POST/DELETE <code>/plans/:id/services</code> · POST/PATCH/DELETE <code>/plans/:id/nominees</code>
Documents	GET/POST <code>/plans/:id/documents</code> · GET <code>/documents/:id/download</code> · DELETE <code>/documents/:id</code>

Services	GET /services (filters) · GET /services/cities · GET /services/:id · provider CRUD
Requests	POST /requests · GET /requests · PATCH /requests/:id/status (validated state machine)
Admin	GET /admin/stats · users · providers/:id/verification · disputes · feedback
Other	categories CRUD · disputes · notifications (read / read-all) · feedback

Service request state machine

pending →accept→ accepted →complete→ completed
 └decline→ declined ←decline┐ (requester may cancel pending / accepted)

10 Security & privacy

Concern	Implementation
Passwords	bcryptjs with 12 salt rounds; never returned in API responses
Sessions	JWT (7-day expiry) in the Authorization header; automatic logout on 401
Field encryption	AES-256-GCM with a random 12-byte IV per value and an auth tag, applied through Mongoose setters/getters
Access control	Owner, nominee (finalized + access on) or 403. Admin endpoints expose counts only, never plan content
Files	Random stored names, type whitelist, 5 MB limit, no public folder; download only after an access check
HTTP hardening	Helmet headers, rate limit on auth (50 requests / 15 min), CORS rules, 1 MB JSON limit, trust-proxy

11 UI / UX design

The subject is emotionally sensitive, so the interface is **calm, respectful and uncluttered**, with no countdowns, aggressive upsells or loud colours.

Element	Decision
Palette	Ivory #FAF7F2 background · Slate #1F2A37 text · Sage #5B7B6A primary · Gold #C8A96A accent · Rose #B5838D warnings
Typography	Fraunces (serif) headings for warmth, Inter body for readability, 16px base size

Components	16px rounded cards, soft shadows, circular icon badges, lotus logo mark, status chips
Language	"Start whenever you feel ready", "when the time comes", "Rest easy"
Accessibility	Labels linked to inputs, focus rings, 42px+ touch targets, keyboard-closable modals, AA contrast
Responsive	Mobile navigation drawer, stacked layouts, scrollable tables, bottom-sheet modals on phones

A detailed prompt for Figma AI / Google Stitch / v0 with low-fidelity wireframes is included in the repository (docs/04_UI_WIREFRAME_PROMPT.md).

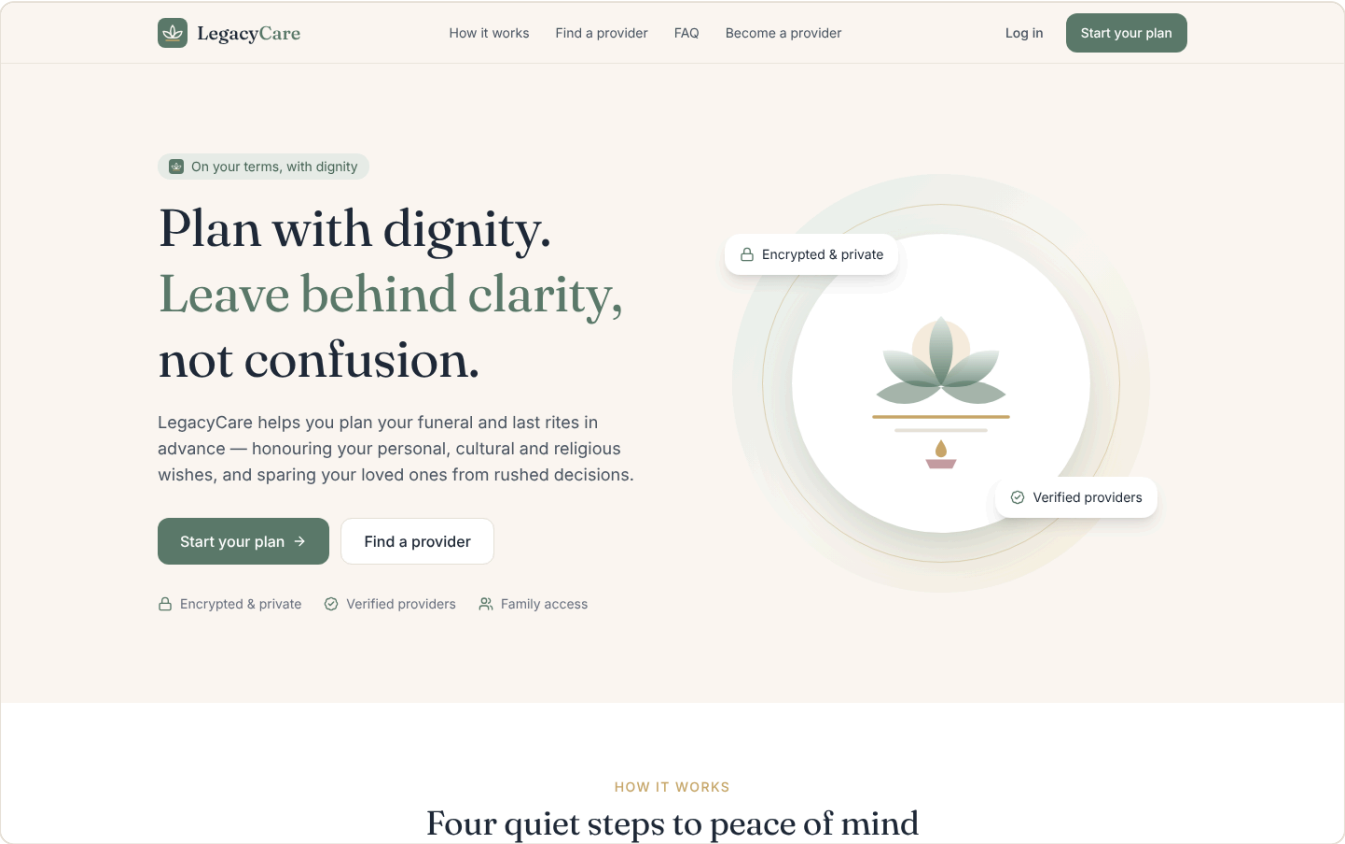


Fig. 1 – Home page (desktop)

12 Application screenshots

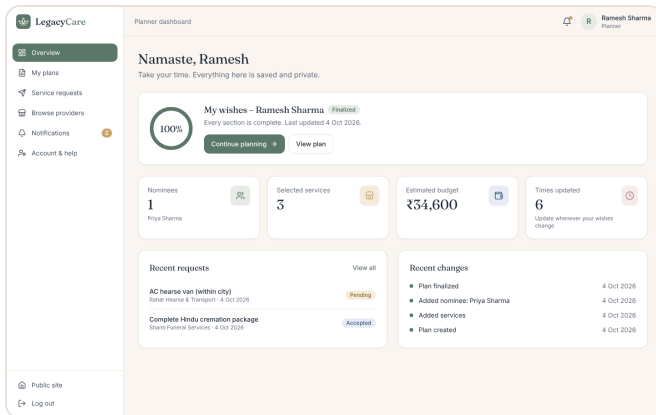


Fig. 2 – Planner overview

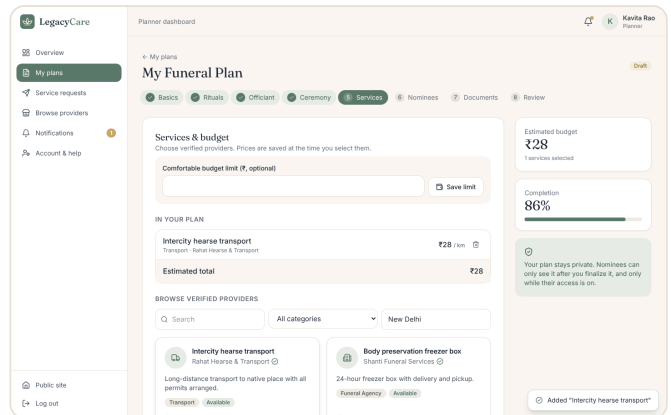


Fig. 3 – Wizard step 5: services & budget

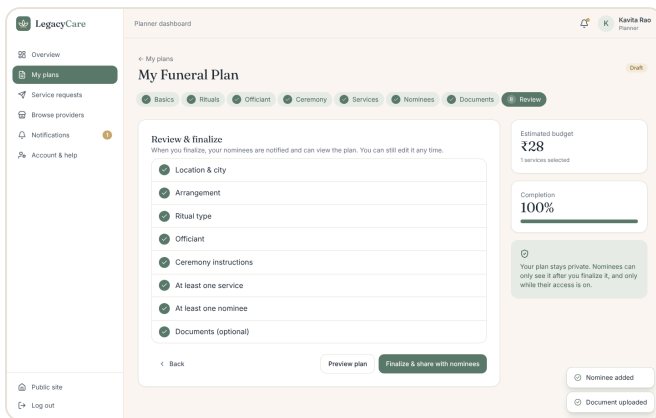


Fig. 4 – Review & finalize

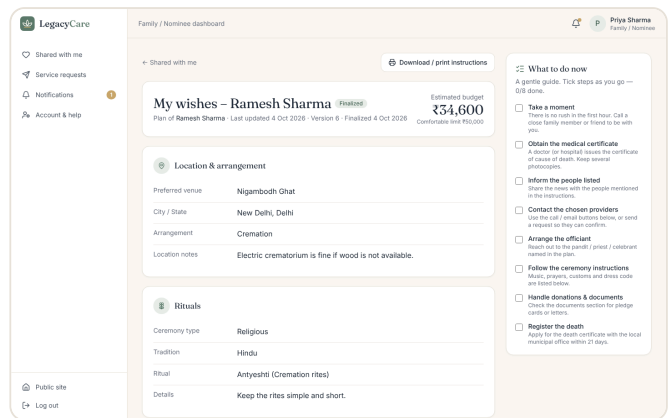


Fig. 5 – Nominee view with guidance checklist

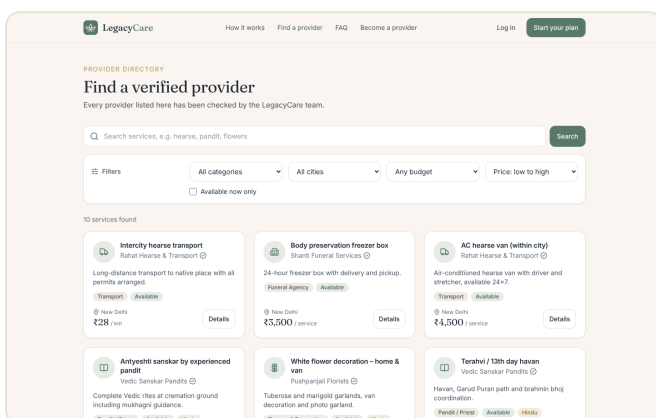


Fig. 6 – Verified provider directory

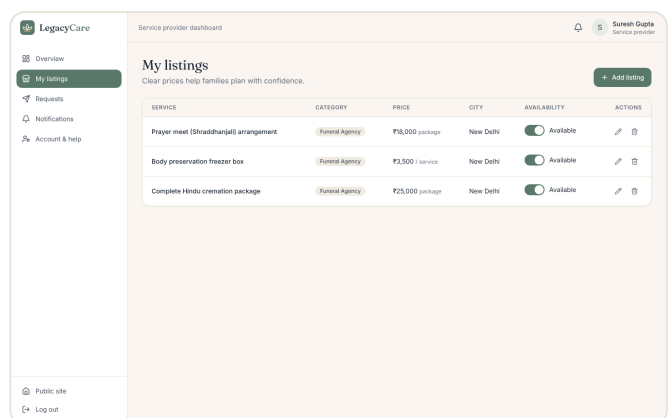


Fig. 7 – Provider listings

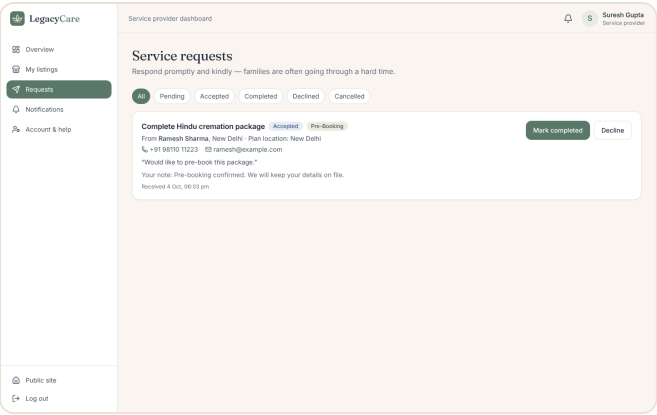


Fig. 8 – Provider request inbox

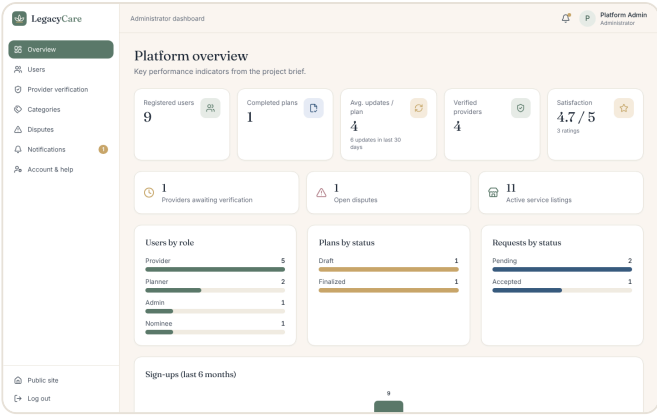


Fig. 9 – Admin KPI dashboard

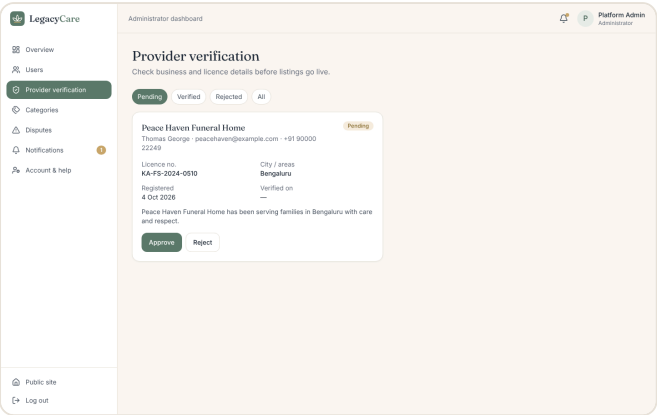


Fig. 10 – Provider verification queue

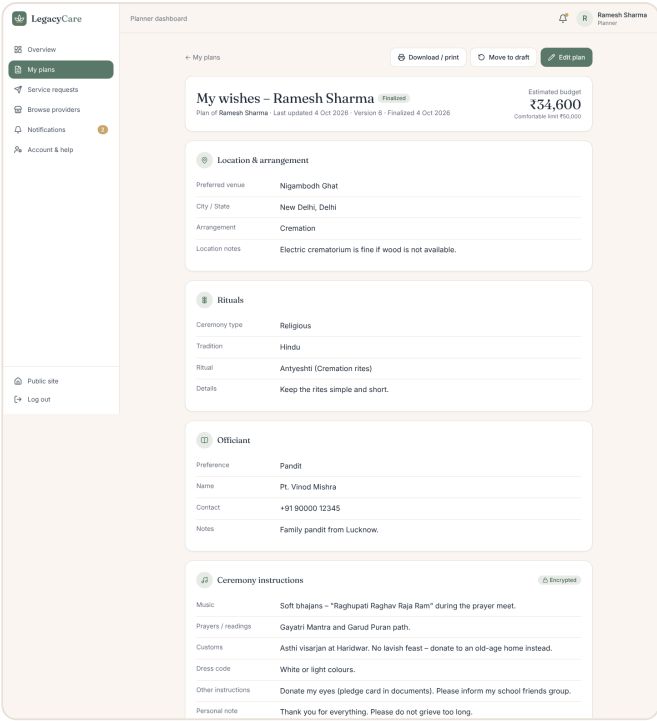


Fig. 11 – Printable plan detail

13 Testing

Two automated suites were run against a freshly seeded database: an **API suite** (Python + requests, 51 checks) and a **browser suite** (Playwright + Chromium) covering every page for all four roles plus a full new-user journey.

Area	Test cases (all passed)
Authentication	Valid login; wrong password → 401; admin self-registration blocked; suspended user blocked → 403
Plans	Finalize blocked when city/ritual/nominee missing; update and version bump; duplicate service → 409; budget recalculated
Encryption	Ceremony field stored as enc:v1:... in MongoDB; decrypted correctly through the API
Nominee access	Blocked on draft; allowed after finalize; phone numbers hidden; access revoke → 403; cannot edit
Documents	Upload OK; .exe rejected; nominee download OK; stranger download → 403
Requests	Create; provider sees it; invalid transition → 400; accept → complete; notifications created
Providers & admin	Pending provider hidden, then visible after verification; ownership enforced; KPIs correct; non-admin → 403; categories CRUD; dispute resolved
UI (Playwright)	All public and dashboard pages render with zero console errors; register → 7 steps → upload → finalize journey passes
Serverless	Vercel handler tested locally: directory, login, upload to /tmp, and blocked foreign origin

14 Deployment

Component	Platform & configuration
Frontend	Vercel CDN: <code>npm run build --prefix client</code> → <code>client/dist</code>
Backend	Vercel serverless function <code>api/index.js</code> wrapping the Express app, with a cached DB connection
Routing	<code>/api/*</code> → function; all other paths → <code>index.html</code> (SPA)
Database	MongoDB Atlas (M0) with network access 0.0.0.0/0 for serverless
Secrets	MONGO_URI, JWT_SECRET, ENCRYPTION_KEY as Vercel environment variables (never committed)

15 Key performance indicators

All five KPIs from the brief are calculated live on the admin dashboard:

Users Registered users by role	Plans Completed (finalized) plans	Updates Avg. plan version and updates in 30 days	Providers Verified provider count	Rating Average satisfaction (1–5)
---	--	--	--	--

16 Challenges & solutions

Challenge	Solution
Sensitive subject matter	Gentle copywriting, calm palette, save-draft everywhere, no pressure tactics
Keeping plans private but shareable	Three-condition nominee access (finalized + email match + access on) and field-level encryption
Cultural diversity	Admin-managed ritual categories per tradition, plus free-text customs fields
Untrusted vendors	Listings hidden until an admin verifies the provider; dispute workflow
MongoDB Atlas querySrv ECONNREFUSED on Windows ISP DNS	Use public DNS resolvers (8.8.8.8 / 1.1.1.1) for the SRV lookup in the DB config
Blank page in VS Code's embedded browser	Safe localStorage wrapper with an in-memory fallback
Newer Chrome returning a Promise from scrollTo, which crashed a React effect	Wrap the effect body in braces so it returns nothing
Mongoose 9 removed next() in async hooks	Rewrote the password hook as a plain async function
File uploads on serverless hosting	Writable /tmp directory on Vercel; cloud storage proposed for production

17 Conclusion & future scope

LegacyCare meets every functional and non-functional requirement in the brief and delivers a deployed, documented, tested MERN application. It turns a difficult, often avoided task into a calm, guided experience, and gives families clarity when they need it most.

Future enhancements

- Android and iOS apps (React Native)
- Email, SMS and WhatsApp reminders and nominee alerts
- Online pre-payment and escrow for provider bookings
- Hindi and regional-language support, with more regional rituals
- Legal will integration and government process assistance
- Emergency notification workflow and cloud document storage (S3 / Cloudinary)

Links

Live application: <https://legacycare-unified-mentor-project.vercel.app>

Source code: <https://github.com/lxkshxyy/LegacyCare-Unified-Mentor-Project>

Demo login (planner): ramesh@example.com / Planner@1234 · Admin: admin@legacycare.in / Admin@1234